

Milestone 4: [20 points] Your own innovations

In addition to the above core features implemented in your cycle-accurate simulator, you are highly encouraged to implement one or more innovative features using your own imagination. These innovative features are open-ended designs and do not have detailed descriptions. You will need to choose, design, implement, and evaluate/test on your own, and demonstrate them during your assigned demo time slot. You may want to take a look at the Venus simulator (emulator) and the popular cycle-accurate gem5 simulator <https://github.com/gem5/gem5>

Below we list a few examples of innovative features that you may consider adding to your simulator design; feel free to add your own innovative features that are not on the list. Typically, each major feature is worth 10 points (except the dual-issue out-of-order pipeline feature and TLB worth 20 points), and each minor feature is worth 5 points (if you add something really minor, you will get fewer points). The following list also gives you a sense of what is considered as a major feature or minor feature. You will need to implement one or more features to get the total 20 points in milestone 4.

Major Features:

- **[Lecture 14] Dual-issue in-order pipeline (10 points)**
Simulate a dual-issue in-order execution pipeline. Each cycle, it fetches and decodes two instructions. As long as there is no structural hazard, control hazard, or data hazard between these two instructions, they can be executed in the same cycle. Note the order to execute the instructions cannot be changed though.
- **[Lecture 14] Dual-issue out-of-order pipeline (20 points)**
Simulate a more advanced dual-issue out-of-order pipeline where two instructions can be issued/executed out of order, as long as their structural hazard, control hazard, or data hazard are resolved. That is, a later instruction can be executed before a prior instruction; however, these instructions must be committed in order. This involves implementing complex scheduling and reservation stations to dynamically determine the instruction execution order, better improving the instruction-level parallelism.
- **[Lecture 15-16] Virtual memory and TLB (20 points)**
Simulate virtual memory, page table, and a TLB (Translation Lookaside Buffer) to cache the virtual-to-physical address translation. To make your life easier, we can assume your program is relatively small such that all data is in memory, i.e., there is no page fault. We can also assume cache accesses always use physical addresses. The TLB should effectively cache recent address translations, reducing the frequency of page table lookups. Efforts must also focus on handling TLB misses (no page faults though) efficiently.
- **[Lecture 12-13] Two-level cache (10 points)**
Simulate a two-level cache for the processor. The first level (L1) cache should be split into separate instruction and data caches. And the second level (L2) cache should be a larger, unified instruction and data cache.
- **[Lecture 9-10] Advanced branch prediction (10 points)**
Simulate a dynamic branch prediction system that utilizes advanced algorithms such as global branch prediction or tournament predictors, which combine several methods for

enhanced prediction accuracy. This feature should effectively address control hazards associated with branch instructions, reducing the impact of branch mispredictions on CPU performance.

Minor Features:

- **[Lecture 12-13] Instruction cache (5 points)**
Simulate an instruction cache specifically designed to fetch instructions efficiently. This cache should be optimized for rapid access to instructions while managing fetch-related hazards effectively through appropriate cache design considerations such as size, block size, associativity, and replacement policy.
- **[Lecture 9-10] 1-bit / 2-bit branch prediction (5 points)**
Implement a simple branch prediction strategy where the 1-bit predictor records the last outcome of the branch (taken or not taken), and the 2-bit predictor employs a small state machine to acknowledge recent trends in branch outcomes. Both should aim to minimize control hazards by reducing the frequency and penalty of branch mispredictions. Note each 1-bit or 2-bit branch predictor is worth 5 points.
- **GUI support (5 points)**
Develop a GUI for the CPU simulator that allows users to configure CPU settings visually, monitor simulation results in real-time, and interactively manage and visualize how data and control hazards are handled within the CPU architecture.

Open Features:

You are also encouraged to explore and implement other additional features to enhance the functionality, performance, or usability of the RISC-V based CPU. You should describe your features with expected functionality, implementation approaches, how it improves the architecture, and the performance you achieve. You may send the TAs a proposal of your features, and we will assign an appropriate number of points for the feature. If a proposal is not received, the number of points will be assigned during the grading of the final project.

Milestone 4 Testing:

For any innovative features you implemented, you should provide comprehensive testcases to demonstrate the functionality and effectiveness of these features. You can include the provided testcases and/or add more testcases based on your features. Basically, your testcases should cover one or more of the following conditions:

- **Functionality Test:** Basic testcases that validate your advanced features. They should operate as expected without disrupting the functionality from previous milestones.
- **Performance Test:** Measure the performance improvements by the new feature. Like execution time improvement from the 2-level cache. Note that you may need to develop more complicated testcases to show the robustness of your implementation.
- **Usability Test:** For features like GUI, provide some testcases to show how your functions enhance the user experience.

- Corner Case Test: Test features with corner cases to show the robustness of your design. For example, you may include errors and data/control hazard in your testcases to verify the correctness of your design.

Milestone 4 Marking:

- Milestone 4 contains 20 points of the project.
- The points earned will be decided by the features that you have completed. Each completed feature must have thorough documentation in the report, well-written testcases and the execution script.
- Half of your maximum points will be given to the demo and report, the other half will be decided by the completion of your features. For example, if you only implement a dual issue in-order pipeline, you can get up to 10 points. 5 points will be given for your demo and report. 5 points will be given for the completion of your features.
- In your report, you should include:
 - The target features that you have implemented
 - A description of the features and demonstrate what impact it may have
 - Report the testcases you have prepared and explain why
 - Clear instructions/commands on how to execute your testcases with the implemented features
 - Report the effectiveness of your features and whether it matches with expectations, and explain why